

One-page master cheatsheet

Read left to right. Replace IMAGE / COMMAND / NAME with your actual values. Linux + modern Compose.

DOCKERFILE + CONTEXT

IMAGE / DIGEST

CONTAINER + CONFIG

01 / Mental model & boundaries

Image: immutable app + user-space files.
Container: process instance + writable layer.
Volume: persistent data, separate lifecycle.
Registry: publishes/pulls image artifacts.
Namespaces: isolation. **cgroups:** limits.
Tag: movable. **Digest:** content identity.
Linux kernel: shared; Desktop uses a VM.
run = new · start = same · exec = extra.

02 / Launch & inspect

BASH

```
docker run -d --name app \
  -p 127.0.0.1:8080:3000 IMAGE
docker ps -a
docker logs -f --tail 100 app
docker exec -it app sh
docker stop app
docker rm app
```

-d: detached · **-it:** interactive terminal.
--rm: disposable · **--init:** init helper.
Assumes the image serves container port 3000.

03 / Build & startup rules

```
docker build -t app:1 .
docker build --pull --no-cache .
```

FROM → WORKDIR → COPY → RUN
USER → EXPOSE → CMD (typical structure)
RUN executes at build; CMD is runtime default.
ENTRYPOINT + CMD = executable + default args.
Prefer exec JSON; last CMD/ENTRYPOINT wins.
Lockfiles first; source last; use .dockerignore.
Multi-stage: copy only runtime artifacts.
Secrets: use mounts, never ARG/ENV.

04 / Compose: use the project

BASH

```
docker compose config
docker compose up -d --build --wait
docker compose ps
docker compose logs -f
docker compose exec api sh
docker compose down
```

up: reconcile; **restart:** reuse config.
down: keeps named volumes; **-v:** deletes them (external volumes are excluded).
Startup order ≠ readiness. Use health + retry.
.env interpolation ≠ automatic injection.

05 / Network & data decisions

H:C = Host:Container.
-p 8080:3000 → host uses port 8080.
Inside app: bind 0.0.0.0, not only localhost.
Peer: **service:container_port** on shared net.
EXPOSE is metadata, not publishing/security.

Durable data → volume.
Edit from host → bind mount.
Temporary memory-backed data → tmpfs.
Stop keeps the writable layer; remove loses it.
Mounts can hide files. Volumes are not backups.

06 / Diagnose & protect

```
docker inspect app
docker stats --no-stream app
docker system df -v
```

Read state/logs before deleting evidence.
137 suggests SIGKILL; check OOMKilled.
Use non-root USER; restrict daemon access.
Drop capabilities; avoid --privileged.
Limit memory/CPU/PIDs; bound logs.
Handle SIGTERM; probe health; test restore.
Unhealthy alone does not restart a container.

NEVER GUESS AT DESTRUCTIVE OPERATIONS

Check the Docker context, exact resource name and backup. A volume prune or compose down -v can delete data you still need.